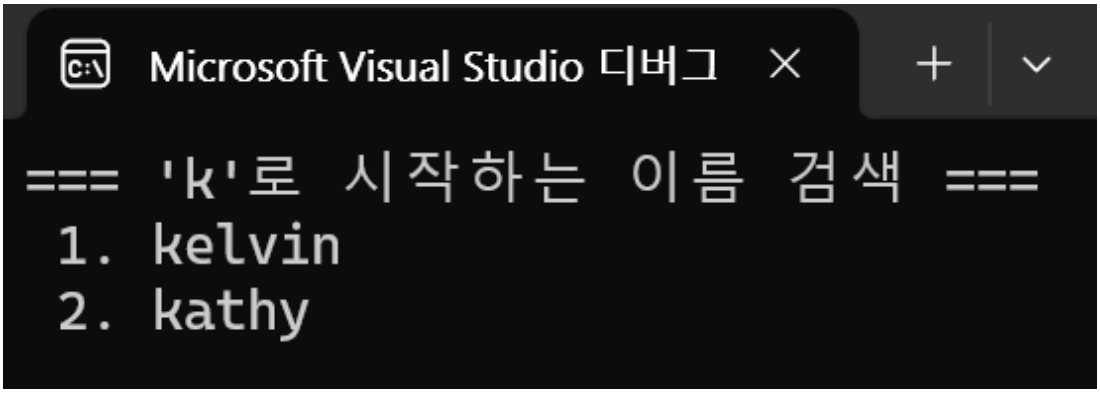
Assignment #3

전기전자공학부 202110939 신주엽

```c
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

typedef struct _node {
    struct _node* next;
} node;

typedef int (*FCMP)(const void*, const void*);

int starts_with_k_cmp(const void* a, const void* b) {
    const char* s = (const char*)a;
    (void)b;  // key를 사용하지 않음
    return (tolower(s[0]) == 'k') ? 0 : 1;
}

void init_list(node** p) {
    *p = (node*)malloc(sizeof(node));
    (*p)->next = NULL;
}

void* llv_insert(void* key, node* base, size_t* num, size_t width, FCMP fcmp) {
    node* t;
    t = (node*)malloc(sizeof(node) + width);
    memcpy(t + 1, key, width);
    t->next = base->next;
    base->next = t;
    (*num)++;
    return t;
}

void* llv_delete(void* key, node* base, size_t* num, size_t width, FCMP fcmp) {
    node* prev = base;
    node* cur = base->next;

    while (cur != NULL) {
        if (fcmp((void*)(cur + 1), key) == 0) {
            prev->next = cur->next;
            free(cur);
            (*num)--;
            return base;  // 삭제 성공
        }
        prev = cur;
        cur = cur->next;
    }
    return NULL;  // 삭제 실패
}
```

```c
void* llv_search(void* key, node* base, size_t* num, size_t width, FCMP fcmp)
{
    node* t;
    t = base->next;
    while (fcmp(t + 1, key) != 0 && t != NULL) t = t->next;
    return (t == NULL ? NULL : t + 1);
}

void main() {
    FILE* fp = fopen("C:₩₩Users₩₩shinj₩₩Desktop₩₩3-1₩₩EmbeddedDataStructure₩₩name.txt", "r");

    node* head;
    int size = 0;
    init_list(&head);

    char buf[100];
    while (fgets(buf, sizeof(buf), fp)) {
        buf[strcspn(buf, "₩n")] = 0; // 개행 제거
        llv_insert(buf, head, &size, strlen(buf) + 1, strcmp);
    }
    fclose(fp);

    printf("=== 'k'로 시작하는 이름 검색 ===₩n");
    void* found;
    int count = 0;

    while ((found = llv_search(NULL, head, &size, sizeof(buf), starts_with_k_cmp)) != NULL) {
        printf("%2d. %s₩n", ++count, (char*)found);
        llv_delete(found, head, &size, strlen((char*)found) + 1, strcmp);
    }

    if (count == 0)
        printf("조건에 맞는 항목이 없습니다.₩n");
}
```



이 프로그램은 연결 리스트를 기반으로 파일에서 입력된 문자열 데이터를 저장하고, 조건에 따라 특정 항목을 탐색하여 출력한 뒤 삭제하는 기능을 수행한다. 삽입, 삭제, 검색 기능은 각각 별도의 함수로 구성되었으며, 사용자 정의 비교 함수를 통해 조건 기반 탐색이 가능하도록 구현되었다. 전체 과정은 순차적인 탐색 알고리즘에 따라 이루어지며, 리스트 구조를 유지하면서 반복적으로 조건에 맞는 데이터를 처리한다.

init_list

연결 리스트의 헤드 노드를 초기화하는 함수로, 더미 노드를 생성하여 head->next부터 실제 데이터가 저장되도록 구조를 설정한다.

llv_insert

새로운 데이터를 리스트 맨 앞에 삽입하는 함수이다. malloc()을 통해 노드를 생성하고, 노드 뒤쪽 메모리 공간에 문자열 데이터를 저장한 후 연결 구조를 업데이트한다.

llv_delete

리스트에서 특정 데이터를 찾아 제거하는 함수이다. fcmp 비교 함수를 이용해 일치하는 항목을 탐색한 후, 리스트에서 제거하고 동적 할당된 메모리를 해제한다. 삭제가 성공하면 리스트 포인터를, 실패하면 NULL을 반환한다.

llv_search

리스트를 순차적으로 탐색하여 주어진 조건을 만족하는 첫 번째 항목을 찾는 함수이다. 일반적인 Sequential Search 알고리즘의 연결 리스트 버전이며, 일치 항목이 발견되면 해당 노드의 데이터 주소를 반환하고, 없으면 NULL을 반환한다.

starts_with_k_cmp

사용자 정의 비교 함수로, 문자열의 첫 글자가 'k' 또는 'K'인지 확인하는 조건 필터 함수이다. 검색 알고리즘(llv_search)에서 비교 기준으로 사용된다.

main 함수 설명

main 함수에서는 먼저 텍스트 파일을 열어 이름을 읽고, 각 이름을 llv_insert()를 통해 연결 리스트에 삽입한다. 이후 'k'로 시작하는 항목을 찾기 위해 llv_search()를 반복적으로 호출하고, 해당 항목이 발견될 경우 출력한 뒤 llv_delete()로 리스트에서 제거한다. 이 작업은 더 이상 조건에 맞는 항목이 없을 때까지 반복되며, 결과적으로 'k'로 시작하는 항목들이 모두 출력되고 리스트에서는 제거된다.